grading_interface

Grading Interface Contract - SPI Master Verification Project

This file is **normative**. Any student submission that violates a **MUST** clause in this file forfeits the corresponding rubric item.

1. File-system contract

Your submission zip must unpack to a single top-level directory named `<name>_<id>/` that contains at minimum:

```
<name>_<id>/
  tb/                    # testbench top-level file(s)
  env/                   # agents, scoreboard, coverage, reference model
  tests/                 # test classes / programs
  sequences/             # stimulus sequences
  assertions/            # SVA bind files (or inline in env/)
  docs/
    test_plan.pdf
    final_report.pdf
    coverage_report.pdf  # exported after `make cov`
  Makefile               # MUST inherit the targets below
  README.md              # how to run, toolchain notes
```

Additional folders are allowed but are ignored by the grader.

The directory layout is mandatory regardless of methodology. An SV-only submission populates the *same* folders but with SystemVerilog programs and modules instead of UVM class libraries (see Section 10).

2. Required Makefile targets (MUST)

Your `Makefile` MUST support exactly the following invocations, with no additional positional arguments required:

Target	Effect
`make compile`	Compile the testbench and the DUT sources given by `DUT_SRCS`
`make run TEST=<name> SEED=<n>`	Run a single test with the given seed

Target	Effect
`make regress`	Run the full regression list, once per `REGRESSION_SEEDS`
`make run_bonus`	Run the mandatory RAL bonus test `ral_hw_reset_test`
`make cov`	Produce `coverage_report.txt` at the repo root
`make clean`	Remove build artefacts

The grader will invoke:

```
make clean
make compile DUT_SRCS="<abs_path_to_spi_core_buggy> \
                       <abs_path_to_apb_regfile_buggy> \
                       <abs_path_to_spi_master_buggy>"
make regress DUT_SRCS=" ... " REGRESSION_SEEDS=<N>
make cov
```

`REGRESSION_SEEDS` is bounded such that **total runs <= 10000**.

DUT_SRCS vs DUT_SRC

The DUT now has three RTL files (`spi_core.sv` , `apb_regfile.sv` , `spi_master.sv`) that together implement the IP. The grader passes the whole list via `DUT_SRCS` .

For backward compatibility the grader's older single-file override `DUT_SRC=<one file>` is still honoured by the template Makefile: if `DUT_SRC` is non-empty it replaces `DUT_SRCS` completely. Students MUST NOT set `DUT_SRC` themselves.

run_bonus

`make run_bonus` runs a test whose name is fixed to `ral_hw_reset_test` (i.e. the Makefile variable `BONUS_TEST` defaults to this). Students who do not attempt the RAL bonus SHOULD keep a stub test under that name that prints `[TEST_SKIPPED] ral_hw_reset_test` ; the grader's RAL detector then counts zero for the bonus and the rest of the rubric is unaffected.

3. Required test names (MUST implement)

Your regression MUST include, at minimum, the following ten tests. Test names are case-sensitive and must match exactly:

1. `sanity_test` - enable, single byte transfer in mode 0, verify RX

2. `reg_access_test` - reset-value + write-read for every R/W register
3. `mode_coverage_test` - all 4 modes x {MSB-first, LSB-first} x {8,16,32}
4. `width_coverage_test` - edge cases at width boundaries
5. `fifo_stress_test` - push/pop near full/empty, back-to-back transfers
6. `interrupt_test` - each of the 5 interrupts: assert, mask, W1C, W1C race
7. `clk_div_corner_test` - DIV = 0, 1, small, large (>= 1024)
8. `loopback_test` - loopback set; external MISO driven nonsense; RX==TX
9. `delay_transfer_test` - DELAY > 0 inserts idle cycles between transfers
10. `error_injection_test` - TX write when full, RX read when empty, illegal width encoding, access to reserved offsets

You MAY add more tests (bonus regression coverage). Any additional test added to `REGRESSION_TESTS` in the Makefile is run by the grader.

4. Log message contract (MUST)

The grader recognises a bug catch **if and only if** at least one of the following appears in the test log, on its own line (leading whitespace allowed):

- `[SCOREBOARD_ERROR] <free text>` - mismatch in predicted vs observed
- `[ASSERTION_ERROR] <assertion name> <free text>` - SVA failure
- `[CHECKER_ERROR] <check name> <free text>` - explicit `$error` from a check
- Any standard Questa/VCS `** Error:` or `UVM_FATAL` / `UVM_ERROR` line from your own code is also accepted.

At the end of each test, your TB MUST print exactly one of:

- `[TEST_PASSED] <test_name>` - zero errors, all expected events seen
- `[TEST_FAILED] <test_name> errors=<n>` - at least one error

The grader uses these lines to count passes/fails independently from the bug-detection count.

5. Coverage contract (MUST)

- `coverage_report.txt` must be produced by `make cov` after a full regression
- The report MUST include total functional and code coverage numbers
- The grader parses the last occurrence of lines of the form `Total Coverage: <pct>%` and `Functional Coverage: <pct>%`
- Coverage gate: functional >= 85% and code (statement/branch) >= 85% on the **golden** RTL. This gate is checked in a separate grader pass using

`DUT_SRC=<path to golden>`

6. Runtime contract (MUST)

- `make regress` must finish in **at most 10000 simulation invocations**. Total runs = `|REGRESSION_TESTS| x REGRESSION_SEEDS`.
- A single test must finish in at most 10 minutes of wall time on the grading machine (reasonable modern 8-core). Timeouts are treated as test failures.
- Tests must be deterministic given `+SEED=<n>` - the grader re-runs failing tests once to verify.

7. UVM bonus contract (OPTIONAL, strictly enforced)

If you attempt the UVM bonus, the grader runs **structural** checks on your source tree. All of the following MUST be observed for the UVM bonus to apply:

1. `tb/tb_top.sv` (or any file under `tb/`) contains `import uvm_pkg::*;` AND calls `run_test()`; the default test name passed to `run_test` MUST be one of your own `uvm_test` subclasses.
2. At least one class in `env/` extends `uvm_env` and defines a `build_phase` that calls `uvm_config_db#(virtual <iface>)::get( ... )` at least once for the APB virtual interface.
3. At least one test in `tests/` registers a `uvm_factory` type override via `set_type_override_by_type` (or `set_inst_override_by_type`) at build time. Comment-only overrides do not count.
4. A `uvm_subscriber`-derived (or `uvm_analysis_imp`-based) coverage collector exists in `env/` and is instantiated somewhere inside the `uvm_env`.

If UVM RAL is used, you additionally qualify for the RAL bonus if ALL of the following hold:

1. A class in `env/` extends `uvm_reg_block` and declares nine `uvm_reg` handles whose names match the register map in the spec (`CTRL`, `STATUS`, `TX_DATA`, `RX_DATA`, `CLK_DIV`, `SS_CTRL`, `INT_EN`, `INT_STAT`, `DELAY`).
2. An `add_hdl_path` / `configure` / `add_hdl_path_slice` call wires the backdoor paths to `dut_wrapper.u_dut.u_regfile.*` (the exposed DUT hierarchy). Backdoor access MUST be demonstrated at runtime - a single `UVM_BACKDOOR` read/write is sufficient.

3. `tests/ral_hw_reset_test.sv` (exactly that filename, class name `ral_hw_reset_test extends uvm_test`) runs `uvm_reg_hw_reset_seq` against the register block and prints `[TEST_PASSED] ral_hw_reset_test` on the golden RTL.
4. `make run_bonus` succeeds on the golden RTL.

The grader performs these checks via:

- Regex/AST search across `tb/`, `env/`, `tests/`.
- An actual `make run_bonus DUT_SRCS=<golden>` invocation. The RAL bonus is awarded **only** if that run prints `[TEST_PASSED] ral_hw_reset_test`.

Partial attempts (e.g. a `uvm_reg_block` declared but never used, no backdoor, or `ral_hw_reset_test` missing) do **not** earn the RAL bonus. They may still earn the UVM bonus if the UVM structural checks pass.

8. Forbidden practices

- Hardcoding the DUT source path (must use `DUT_SRC`)
- Short-circuiting tests based on `+define+FAULTY` or similar: the grader will compile with `+define+SIM` only
- Detecting the grader by hostname/username/env var
- Consuming more than 10 GB of disk or RAM during a single test
- Calling external services at test time

Violations void the bug-detection score.

9. Submission checklist

- [ ] Zip named `<name>_<id>.zip`
- [ ] `make compile` succeeds with the default `DUT_SRCS` (golden RTL)
- [ ] `make regress` finishes under the 10000-run cap
- [ ] `coverage_report.txt` produced and >= 85% on golden RTL
- [ ] `docs/test_plan.pdf`, `docs/final_report.pdf`, `docs/coverage_report.pdf` all present
- [ ] `README.md` lists simulator version, any non-default compile flags
- [ ] No binary simulator output files checked in (`*.wlf`, `work/`, etc.)
- [ ] If UVM bonus attempted: `make run_bonus` passes on golden and the RAL structural checks in Section 7 all hold.

10. SystemVerilog-only track (mandatory baseline)

UVM is **optional**. If you choose to stay in plain SystemVerilog, your submission MUST still fit the Section 1 layout. The grader does not treat SV-only and UVM submissions differently for the base rubric - only the UVM and RAL bonuses are off the table.

Recommended organisation for an SV-only submission:

```
tb/
  tb_top.sv              # module (not a class) instantiating dut_wrapper
  apb_master_bfm.sv      # `program` or `module` that drives apb_if
  spi_slave_bfm.sv       # module that drives/monitors spi_if.slave
env/
  ref_model.sv           # `module` or `program` implementing the predictor
  scoreboard.sv          # pure SV scoreboard (queues + $error on mismatch)
  coverage.sv            # covergroups on APB, SPI, interrupt events
tests/
  sanity_test.sv         # `program` automatic; selected by +TESTNAME= ...
  reg_access_test.sv
  mode_coverage_test.sv
   ...
sequences/
  stim_lib.sv            # reusable randomisable transaction classes
assertions/
  spi_sva.sv             # module bound to dut_wrapper via `bind`
```

`tb/tb_top.sv` MUST look roughly like this skeleton (full version is shipped in `harness/examples/sv_only/`):

```
`timescale 1ns/1ps
module tb_top;
  // clk / rst / interfaces
  bit PCLK = 0; always #5 PCLK = ~PCLK;
  bit PRESETn;
  apb_if apb (PCLK, PRESETn);
  spi_if spi ();

  dut_wrapper u_wrap (.apb(apb.slave), .spi(spi), .PCLK(PCLK),
.PRESETn(PRESETn));

  // bind assertions (use the *instance path* of your dut_wrapper instance,
  // here `u_wrap`, NOT the bare module-type `dut_wrapper`)
  bind u_wrap.u_dut.u_regfile spi_sva u_sva (.*);

  // test dispatch: +TESTNAME=<name> selects which program to spawn
  string testname;
```

```
  initial begin
    PRESETn = 0;
    #50 PRESETn = 1;
    if (!$value$plusargs("TESTNAME=%s", testname)) testname = "sanity_test";
    case (testname)
      "sanity_test"          : sanity_test         :: run(apb, spi);
      "reg_access_test"      : reg_access_test     :: run(apb, spi);
      "mode_coverage_test"   : mode_coverage_test  :: run(apb, spi);
      // ... all ten required names ...
      default                : $fatal(1, "Unknown test %s", testname);
    endcase
    $display("[TEST_PASSED] %s", testname);
    $finish;
  end
endmodule
```

Key contract points for SV-only:

- The grader drives `+TESTNAME=<name>` AND `+UVM_TESTNAME=<name>`. Your TB MUST honour at least one of them; the template Makefile already passes both.
- Every test MUST end with exactly one `[TEST_PASSED]` or `[TEST_FAILED]` line as in Section 4.
- Scoreboards MUST signal mismatches via `[SCOREBOARD_ERROR] ...` lines (Section 4). `$error` is also accepted.
- SVA MUST live in a file that is bound into the DUT (e.g. via `bind u_wrap.u_dut.u_regfile ...` or at the `u_wrap.u_dut.u_core` level). Inline assertions inside the DUT are not counted because you cannot modify the DUT.
- `REGRESSION_TESTS` in the Makefile MUST list all ten required test names (Section 3). You may add more.

A ready-to-clone SV-only scaffold lives at `harness/examples/sv_only/` (shipped with the starter kit). Start from there if you are not using UVM.